

LAB MANUAL — NGINX WITH ENVIRONMENT VARIABLES

Injecting runtime environment variables into HTML using Docker + NGINX + Kubernetes

1. Assumptions & Dependencies

Before starting the lab, ensure the following assumptions are true.

1.1 System Requirements

Component Requirement

OS	Windows / Linux / macOS
Docker	Docker Engine $\geq$ 20.x
Kubernetes	Minikube / Kind / K3s / Cloud cluster
kubectl	Installed & configured
Text Editor	VS Code / nano / vim

1.2 Network & Ports

- Port **8080** will be used on the host system.
 - Kubernetes NodePort will expose port **30080**.
-

1.3 Docker & Kubernetes Knowledge

You should understand:

- Basic Dockerfile usage
 - Entrypoint scripts
 - Kubernetes Pod & Service deployments
-

2. Lab Objective

You will build a **custom NGINX Docker image** that:

- Reads environment variables **WELCOME_MSG** and **ENV**
 - Injects them into an HTML template
 - Serves the generated HTML via NGINX
 - Deploys the container into Kubernetes with its own environment values
-

3. Final Folder Structure

Create the following directory:

nginx-env-demo/

├── Dockerfile

├── entrypoint.sh

└── index.html.template

4. Step-by-Step Lab Instructions

Step 1 — Create Dockerfile

Create Dockerfile inside the project directory.

FROM nginx:alpine

COPY index.html.template /usr/share/nginx/html/index.html.template

COPY entrypoint.sh /docker-entrypoint.d/entrypoint.sh

RUN chmod +x /docker-entrypoint.d/entrypoint.sh

EXPOSE 80

What this does

- Uses lightweight nginx:alpine

- Adds template HTML to NGINX default web dir
 - Adds entrypoint script to auto-run during container startup
 - Exposes port 80
-

Step 2 — Create entrypoint.sh

This script injects environment variables into the HTML template.

```
#!/bin/sh

set -e

# Replace placeholders with environment variables at runtime
envsubst '${WELCOME_MSG}' \
  < /usr/share/nginx/html/index.html.template \
  > /usr/share/nginx/html/index.html
```

Important Dependency

- Requires **envsubst** (already available in Alpine via gettext package, included in NGINX image)
-

Step 3 — Create HTML Template

File: index.html.template

```
<!DOCTYPE html>

<html>

<head>

  <title>Welcome to NGINX</title>

</head>
```

```
<body>
  <h1>${WELCOME_MSG}</h1>
  <p>Environment: ${ENV}</p>
</body>
</html>
```

Step 4 — Build the Docker Image

Run:

```
docker build -t nginx-env-demo .
```

Step 5 — Run the Container with Env Variables

```
docker run -d \
  -p 9080:80 \
  -e WELCOME_MSG="Hello from Dockerized NGINX!" \
  -e ENV="Production" \
  nginx-env-demo
```

Verify Output

Visit:

👉 <http://<vmip or localhost>:9080>

If you are running on the vm , use the VM ip .

You should see:

Hello from Dockerized NGINX!

Environment: Production

6.1 Prerequisites for Docker Hub Push

Requirement	Description
Docker Hub Account	Must be created at https://hub.docker.com
Docker CLI	Logged in to Docker Hub
Internet Access	Required to push and pull images

6.2 Login to Docker Hub

Run the following command:

```
$ docker login
```

Enter:

- **Docker Hub Username**
- **Docker Hub Password / Token**

Verify login:

```
$ docker info | grep Username
```

6.3 Tag the Local Image for Docker Hub

Docker Hub requires the image name format:

<dockerhub-username>/<image-name>:<tag>

Example:

```
$ docker tag nginx-env-demo mydockerid/nginx-env-demo:v1
```

Check tagged image:

```
$ docker images
```

6.4 Push Image to Docker Hub

```
$ docker push mydockerid/nginx-env-demo:v1
```

✅ After this, your image is now **publicly available for Kubernetes to pull**.

6.5 Verify Image on Docker Hub

Open in browser:

<https://hub.docker.com/r/mydockerid/nginx-env-demo>

Confirm:

- Image tag exists
- Visibility is **Public**

7. Kubernetes Deployment

Step 7.1 — Create the deployment YAML

File: nginx-env-demo-deploy.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-env-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx-env-demo
  template:
    metadata:
      labels:
        app: nginx-env-demo
    spec:
      containers:
        - name: nginx
          image: mydockerid/nginx-env-demo:v1
          ports:
            - containerPort: 80
```

```
env:
  - name: WELCOME_MSG
    value: "Hello from Kubernetes NGINX!"
  - name: ENV
    value: "Staging"
```

Step 7.2 — Create the service YAML

File: nginx-env-demo-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-env-demo
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30308
  selector:
    app: nginx-env-demo
```

Step 8 — Apply Kubernetes Manifests

```
kubectl apply -f nginx-env-demo-deploy.yaml
```

```
kubectl apply -f nginx-env-demo-service.yaml
```

Step 9 — Verify Deployment

Check Pod:

```
kubectl get pods
```

Check Service:

```
kubectl get svc nginx-env-service
```

Access Application:

curl http://<minikube-ip>:30308

Get Minikube IP:

minikube ip

10. Troubleshooting Guide

Issue	Fix
Pod stuck in CrashLoopBackOff	Check logs → kubectl logs nginx-env-demo
HTML not updated	Ensure envsubst placeholders match variable names
Cannot access NodePort	Ensure firewall allows port 30080
Image not found	Run minikube image load nginx-env-demo
